

PM2 vs Warden — Side-by-Side Comparison

Warden is a self-hosted process manager built as a PM2-equivalent for Node 18+. This document compares the two tools feature-by-feature so you can decide which fits your stack — or migrate from PM2 with minimal friction.

At a glance

	PM2	Warden
Purpose	Production process manager for Node.js and beyond	Self-hosted PM2 alternative with built-in telemetry
Language	JavaScript	TypeScript
License	AGPL-3.0	MIT
Node requirement	Node 12+	Node 18+
Install	<code>npm install -g pm2</code>	<code>npm install -g @roshan-gamage/warden</code>
CLI binary	<code>pm2</code>	<code>warden</code>
Daemon IPC	Unix socket / TCP	Unix domain socket (named pipe on Windows)
Config home	<code>~/.pm2/</code>	<code>~/.warden/</code>
HTTP API	Optional (via PM2 Plus / custom)	Built-in on <code>127.0.0.1:9615</code>
Prometheus	Via third-party modules	Native <code>/metrics</code> endpoint
Metrics storage	External (PM2 Plus cloud)	Local SQLite (24-hour rolling window)
Cloud dashboard	PM2 Plus (paid)	None — self-hosted only

Feature comparison

Feature	PM2	Warden	Notes
Fork mode (any process)	✓	✓	Node, Python, Ruby, binaries, shell scripts
Cluster mode (Node.js)	✓	✓	Multi-instance, shared port
Auto-restart on crash	✓	✓	Exponential backoff in Warden
Max restarts / min uptime	✓	✓	Stability gate before resetting crash counter
Zero-downtime reload	✓	✓	Cluster: rolling reload; fork: graceful restart
Runtime scale	✓	✓	<code>pm2 scale</code> ↔ <code>warden scale</code>
File watch → reload	✓	✓	chokidar-based in Warden
Cron restart	✓	✓	Scheduled restarts via cron expression
Memory limit restart	✓	✓	e.g. <code>--max-memory-restart 150M</code>
Per-process logs	✓	✓	stdout / stderr to separate files
Log streaming	✓	✓	<code>pm2 logs</code> ↔ <code>warden logs</code>
Log rotation	✓ (module)	✓ (built-in)	Warden rotates on size; PM2 uses <code>pm2-logrotate</code> module
Log flush	✓	✓	<code>pm2 flush</code> ↔ <code>warden flush</code>
Save / resurrect	✓	✓	<code>dump.pm2</code> ↔ <code>dump.json</code>
Startup hook	✓	✓	systemd, launchd; PM2 also supports Windows service via extra package
Ecosystem config	✓	✓	<code>ecosystem.config.js</code> — largely compatible field names
Live terminal dashboard	✓ <code>pm2 monit</code>	✓ <code>warden monit</code>	Auto-refreshing CPU / memory table
Process list / describe	✓	✓	<code>pm2 list</code> / <code>pm2 show</code> ↔ <code>warden list</code> / <code>warden describe</code>
Reset restart counters	✓	✓	<code>pm2 reset</code> ↔ <code>warden reset</code>
Graceful daemon shutdown	✓ <code>pm2 kill</code>	✓ <code>warden kill</code>	Stops all apps and the god process
Environment variables	✓	✓	CLI <code>-e KEY=VAL</code> or config <code>env</code> block
Custom interpreter	✓	✓	<code>--interpreter python3</code> etc.

Feature	PM2	Warden	Notes
Merge logs	✓	✓	Single combined stdout/stderr file
Kill timeout	✓	✓	Graceful shutdown before SIGKILL
HTTP REST API	⚠ Limited	✓ Built-in	Warden: <code>/processes</code> , <code>/summary</code> , <code>/health</code>
Prometheus metrics	⚠ Via modules	✓ Native	Warden exposes <code>warden_process_*</code> gauges
Time-series metrics DB	✗	✓	SQLite, 5 s sampling, 24 h retention
PM2 Plus cloud monitoring	✓ (paid)	✗	Alerts, web UI, multi-server — PM2 only
Module ecosystem	✓	✗	<code>pm2-logrotate</code> , <code>pm2-server-monit</code> , etc.
Deploy automation	✓ <code>pm2 deploy</code>	✗	Git-based deployment workflow — PM2 only
Process attach / debug	✓ <code>pm2 attach</code>	✗	Attach to process stdio — PM2 only
Custom triggers / hooks	✓ <code>pm2 trigger</code>	✗	Event-driven custom actions — PM2 only
Docker official image	✓	✗	PM2 publishes maintained Docker images
Windows service	✓ (via <code>pm2-windows-service</code>)	⚠ Partial	Warden runs on Windows; no native service installer
JSON machine output	✓ <code>pm2 jlist</code>	✗	Scriptable JSON output — PM2 only
In-place PM2 upgrade	✓ <code>pm2 update</code>	✗	Reinstall via npm for Warden
Multi-server fleet mgmt	✓ (PM2 Plus)	✗	Centralized fleet view — PM2 Plus only

Legend: ✓ Supported · ⚠ Partial or requires extra setup · ✗ Not supported

CLI command mapping

Most day-to-day PM2 commands have a direct Warden equivalent:

PM2	Warden	Description
<code>pm2 start app.js</code>	<code>warden start app.js</code>	Start a process
<code>pm2 start ecosystem.config.js</code>	<code>warden start ecosystem.config.js</code>	Start from ecosystem file
<code>pm2 stop <id name></code>	<code>warden stop <id name></code>	Stop a process
<code>pm2 restart <id name></code>	<code>warden restart <id name></code>	Hard restart
<code>pm2 reload <id name></code>	<code>warden reload <id name></code>	Zero-downtime reload (cluster)
<code>pm2 delete <id name></code>	<code>warden delete <id name></code>	Stop and remove from registry
<code>pm2 scale <name> <n></code>	<code>warden scale <name> <n></code>	Scale cluster workers
<code>pm2 list / pm2 ls</code>	<code>warden list / warden ls</code>	Process table
<code>pm2 show <id name></code>	<code>warden describe <id name></code>	Detailed process info
<code>pm2 logs [id]</code>	<code>warden logs [id]</code>	Stream logs
<code>pm2 flush [id]</code>	<code>warden flush [id]</code>	Truncate log files
<code>pm2 monit</code>	<code>warden monit</code>	Live dashboard
<code>pm2 save</code>	<code>warden save</code>	Persist process list
<code>pm2 resurrect</code>	<code>warden resurrect</code>	Restore saved processes
<code>pm2 startup</code>	<code>warden startup</code>	Install boot-time service
<code>pm2 unstartup</code>	<code>warden unstartup</code>	Remove boot-time service
<code>pm2 reset <id name></code>	<code>warden reset <id name></code>	Reset restart counters
<code>pm2 kill</code>	<code>warden kill</code>	Shut down daemon and all apps
—	<code>warden ping</code>	Check daemon status (no PM2 equivalent)

PM2 commands with no Warden equivalent

PM2 command	What it does
<code>pm2 deploy</code>	Git-based deployment to remote servers
<code>pm2 attach</code>	Attach to a running process's stdio
<code>pm2 trigger</code>	Fire custom event hooks
<code>pm2 sendSignal</code>	Send arbitrary signals to a process
<code>pm2 jlist / pm2 prettylist</code>	JSON output for scripting
<code>pm2 env <id></code>	Show resolved environment for a process
<code>pm2 update</code>	In-place upgrade of PM2 itself
<code>pm2 install <module></code>	Install PM2 modules

start options comparison

Option	PM2	Warden
<code>--name</code>	✓	✓ <code>-n, --name</code>
<code>--instances / -i</code>	✓	✓ <code>-i, --instances (supports max)</code>
<code>--exec-mode</code>	✓	✓ <code>--exec-mode fork cluster</code>
<code>--interpreter</code>	✓	✓ <code>--interpreter</code>
<code>--node-args</code>	✓	✓ <code>--node-args</code>
<code>--cwd</code>	✓	✓ <code>--cwd</code>
<code>--env / -e</code>	✓	✓ <code>-e, --env KEY=VAL</code>
<code>--watch</code>	✓	✓ <code>--watch</code>
<code>--max-memory-restart</code>	✓	✓ <code>--max-memory-restart</code>
<code>--cron / cron_restart</code>	✓	✓ <code>--cron</code>
<code>--max-restarts</code>	✓	✓ <code>--max-restarts</code>
<code>--min-uptime</code>	✓	✓ <code>--min-uptime</code>
<code>--restart-delay</code>	✓	✓ <code>--restart-delay</code>
<code>--no-autorestart</code>	✓	✓ <code>--no-auto-restart</code>
<code>--merge-logs</code>	✓	✓ <code>--merge-logs</code>
<code>--kill-timeout</code>	✓	✓ <code>--kill-timeout</code>
<code>--log-date-format</code>	✓	⚠ Config only
<code>--out / --error</code>	✓	⚠ Config only (<code>out_file</code> , <code>err_file</code>)
<code>--ignore-watch</code>	✓	⚠ Config only (<code>ignore_watch</code>)
<code>--wait-ready</code>	✓	✗
<code>--listen-timeout</code>	✓	✗
<code>--shutdown-with-message</code>	✓	✗

Ecosystem config compatibility

Warden reads the same ecosystem file formats PM2 uses (`ecosystem.config.js` , `ecosystem.config.json` , and other `*.config.js` files). Field names follow PM2 conventions:

```
// Works with both PM2 and Warden
module.exports = {
  apps: [
    {
      name: 'api',
      script: './src/api.js',
      instances: 2,
      exec_mode: 'cluster',
      env: { NODE_ENV: 'production', PORT: '3000' },
      max_memory_restart: '500M',
      watch: ['./src'],
      ignore_watch: ['node_modules'],
      cron_restart: '0 3 * * *',
      interpreter: 'node',
      max_restarts: 16,
      min_uptime: '1s',
      autorestart: true,
      merge_logs: true,
      kill_timeout: 5000,
    },
  ],
};
```

Migration tip: If you already have a PM2 ecosystem file, you can usually point Warden at it unchanged:

```
# Before
pm2 start ecosystem.config.js

# After
warden start ecosystem.config.js
```

File locations

Purpose	PM2	Warden
Home directory	~/.pm2/	~/.warden/
Saved process list	~/.pm2/dump.pm2	~/.warden/dump.json
Daemon PID	~/.pm2/pm2.pid	~/.warden/daemon.pid
IPC socket	~/.pm2/rpc.sock / pub.sock	~/.warden/daemon.sock
Logs	~/.pm2/logs/	~/.warden/logs/<name>-<id>/
Metrics	External (PM2 Plus)	~/.warden/metrics.db (SQLite)

Observability

PM2

- **Terminal:** `pm2 monit`, `pm2 list` with inline CPU/memory
- **Cloud:** PM2 Plus — web dashboard, alerts, multi-server (paid subscription)
- **Prometheus:** Requires community modules or custom integration
- **REST API:** Limited; full API tied to PM2 Plus

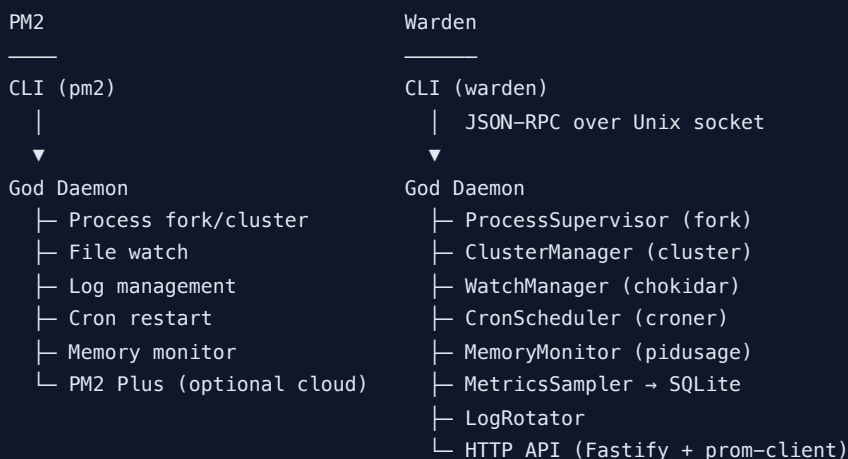
Warden

- **Terminal:** `warden monit`, `warden list`, `warden describe`
- **HTTP API:** Built-in on port 9615 — `/processes`, `/processes/:id/metrics?range=1h`, `/summary`, `/health`
- **Prometheus:** Native `/metrics` endpoint with `warden_process_*` gauges
- **Time-series:** SQLite database with 5-second sampling and 24-hour retention

Example Prometheus scrape config for Warden:

```
scrape_configs:
  - job_name: warden
    static_configs:
      - targets: ['127.0.0.1:9615']
```

Architecture



Both use a long-running **god daemon** that outlives the CLI. The CLI connects, sends commands, and exits — managed processes keep running.

When to choose PM2

- You need **PM2 Plus** cloud monitoring, alerts, or multi-server fleet management
- You rely on `pm2 deploy` for Git-based deployment workflows
- You use the **PM2 module ecosystem** (`pm2-logrotate` , custom modules)
- You need `pm2 attach` , `pm2 trigger` , or **JSON output** (`pm2 jlist`) for automation
- You want an **official Docker image** with PM2 pre-installed
- You need **Windows service** integration out of the box

When to choose Warden

- You want a **fully self-hosted** solution with no cloud dependency or subscription
- You need **native Prometheus** metrics and a **built-in HTTP API** without extra modules
- You want **local time-series metrics** (SQLite) for debugging and short-term analysis
- You prefer an **MIT-licensed**, TypeScript codebase you can fork and extend
- You're running a **single server or small fleet** and terminal + Prometheus/Grafana is enough
- You're migrating from PM2 and your workflow is: start → logs → monit → save → startup

Quick migration guide (PM2 → Warden)

```
# 1. Save your current PM2 process list (for reference)
pm2 save
pm2 jlist > pm2-backup.json

# 2. Stop PM2 (optional – they can coexist on different sockets)
pm2 kill

# 3. Install Warden
npm install -g @roshan-gamage/warden

# 4. Start your apps with the same ecosystem file
warden start ecosystem.config.js

# 5. Persist and enable boot startup
warden save
warden startup    # follow printed instructions for systemd / launchd

# 6. Verify
warden list
warden ping
curl http://127.0.0.1:9615/health
```

If you start processes individually instead of via an ecosystem file, recreate them with equivalent `warden start` flags (see CLI mapping above), then `warden save` .

Summary

Dimension	Winner
Core process management (fork, cluster, restart, logs)	Tie — feature parity for everyday use
Ecosystem config compatibility	Tie — same file format
Built-in observability (API, Prometheus, metrics DB)	Warden
Cloud monitoring & fleet management	PM2 (PM2 Plus)
Module / plugin ecosystem	PM2
Deployment automation	PM2 (<code>pm2 deploy</code>)
Licensing & self-hosting	Warden (MIT, no cloud lock-in)
Maturity & community	PM2 — larger user base, longer track record

Warden covers the PM2 workflow most teams actually use — start, supervise, log, monitor, persist, and boot — while adding first-class telemetry. PM2 remains the better choice when you need cloud dashboards, deployment tooling, or the module ecosystem.